

Patrones de Diseño: Strategy

Este patrón te permite definir una familia de algoritmos (en tu caso, los métodos de 2FA), encapsular cada uno en su propia clase y hacer que sus objetos sean **intercambiables**.

El "jefe" (la clase de Lógica de Login) no sabe *cómo* se hace la validación 2FA, solo sabe que tiene un "ayudante" (la Estrategia) que sabe cómo hacerlo.

Los Componentes del Patrón

1. **El Contexto (Context):** La clase que *usa* la estrategia. Es el `LoginManager` o `AuthService`. Contiene una referencia a un objeto de la estrategia, pero solo se comunica con él a través de la interfaz común.
 2. **La Interfaz de Estrategia (Strategy Interface):** Define el "contrato" que todas las estrategias deben cumplir. Es un `interface` de PHP que define los métodos comunes, como `verify()` o `requestChallenge()`.
 3. **Las Estrategias Concretas (Concrete Strategies):** Son las clases que implementan la interfaz. Cada una representa un método de 2FA: `TotpStrategy` (Authenticator), `SmsStrategy`, `EmailCodeStrategy`, etc.
-

Aplicación 1: Múltiples Métodos de 2FA (El encare más limpio)

Este es el uso más "puro" para tu petición. El usuario ya ingresó su contraseña y ahora debe elegir su *segundo factor*.

1. La Interfaz de Estrategia

Definimos el contrato. Todas las estrategias 2FA deben poder validar un código.

PHP

// Strategy Interface

```
interface TwoFactorStrategy {
```

```
    /**
```

```
     * Valida el código proporcionado por el usuario.
```

```
     * @param User $user El usuario que intenta loguearse.
```

```
     * @param string $code El código 2FA (ej: "123456").
```

```
     * @return bool True si el código es válido.
```

```
    */
```

```
    public function verify(User $user, string $code): bool;
```

```
}
```

2. Las Estrategias Concretas

Implementamos una clase para cada método.

Estrategia A: TOTP (Google Authenticator) Esta estrategia necesita una librería para verificar el código TOTP contra el "secreto" del usuario.

PHP

// Concrete Strategy 1

```
class TotpStrategy implements TwoFactorStrategy {

    public function __construct(private TotpLibrary $totpService) {
        // $totpService sería una librería como 'pragmarx/google2fa'
    }

    public function verify(User $user, string $code): bool {
        // La lógica específica para validar un código TOTP
        return $this->totpService->verify(
            $user->getTotpSecret(),
            $code
        );
    }
}
```

Estrategia B: Código por Email (Como alternativa al TOTP)

PHP

// Concrete Strategy 2

```
class EmailCodeStrategy implements TwoFactorStrategy {

    public function __construct(private MailService $mailer) {}

    // Esta estrategia podría tener un método extra para ENVIAR el código
    public function requestChallenge(User $user): void {
        $code = rand(100000, 999999);
        // Guardar $code en la sesión o BBDD con una expiración
        $_SESSION['2fa_code'] = $code;
        $_SESSION['2fa_expiry'] = time() + 300; // 5 minutos

        $this->mailer->send($user->email, "Tu código 2FA", "El código es: $code");
    }

    public function verify(User $user, string $code): bool {
        // Lógica específica: comparar con el código en sesión
        if (!isset($_SESSION['2fa_code']) || $_SESSION['2fa_expiry'] < time()) {
            return false; // Expiró o no existe
        }
    }
}
```

```

        return $_SESSION['2fa_code'] === $code;
    }
}

```

3. El Contexto

El **LoginManager** decide qué estrategia usar, probablemente basándose en la configuración del usuario.

PHP

// Context

```

class LoginManager {
    private TwoFactorStrategy $strategy;

    // El contexto recibe la estrategia que debe usar
    public function setStrategy(TwoFactorStrategy $strategy) {
        $this->strategy = $strategy;
    }

    public function completeLogin(User $user, string $code): bool {
        if (!$this->strategy) {
            throw new Exception("No se ha definido una estrategia 2FA.");
        }

        // El Contexto delega el trabajo.
        // No sabe si está llamando a TOTP o a Email, solo llama a verify().
        $isValid = $this->strategy->verify($user, $code);

        if ($isValid) {
            echo "¡Bienvenido, {$user->name}!";
            // ... crear la sesión de usuario
            return true;
        }

        echo "Código 2FA incorrecto.";
        return false;
    }
}

```

Uso:

PHP

```

$user = /* ... obtener usuario de la BBDD ... */;
$loginManager = new LoginManager();

```

// El usuario eligió TOTP en su perfil

```

if ($user->preferred_2fa === 'totp') {
    $totpService = new TotpLibrary();
    $loginManager->setStrategy(new TotpStrategy($totpService));
}
// El usuario eligió Email
else if ($user->preferred_2fa === 'email') {
    $mailer = new MailService();
    $loginManager->setStrategy(new EmailCodeStrategy($mailer));
}

// ...
// El controlador recibe el código "123456" del formulario
$codigo_usuario = $_POST['2fa_code'];
$loginManager->completeLogin($user, $codigo_usuario);

```

Aplicación 2: Estrategias de Login *Primarias* (Incluyendo Google/GitHub)

Tu pregunta mezcla 2FA (TOTP) con OAuth (Google, GitHub). **OAuth no es 2FA**, es un método de *autenticación primario* completo.

Pero la buena noticia es que **el patrón Strategy también sirve perfectamente para esto**.

En este caso, la estrategia es el *método de login completo*.

1. La Interfaz de Estrategia

PHP

```

// Strategy Interface
interface AuthStrategy {
    /**
     * Intenta autenticar a un usuario.
     * @param array $credentials Datos del request (ej: $_POST o $_GET)
     * @return User|null El usuario si es exitoso, o null.
     */
    public function authenticate(array $credentials): ?User;
}

```

2. Las Estrategias Concretas

PHP

// Estrategia A: Login clásico

```

class PasswordStrategy implements AuthStrategy {
    public function authenticate(array $credentials): ?User {
        $user = User::findByEmail($credentials['email']);
        if ($user && password_verify($credentials['password'], $user->password)) {
            return $user;
        }
    }
}

```

```

    }
    return null;
}
}

// Estrategia B: Google (OAuth)
class GoogleOAuthStrategy implements AuthStrategy {
    public function authenticate(array $credentials): ?User {
        $code = $credentials['code']; // El código que Google nos da en la URL
        // ... Lógica compleja de OAuth ...
        // 1. Intercambiar $code por un Access Token con Google
        // 2. Pedir la información del usuario a Google con el Access Token
        // 3. Buscar en BBDD si ya tenemos ese usuario, o crearlo
        $userData = $this->googleApiService->getUserProfile($code);
        $user = User::findOrCreateFromGoogle($userData);
        return $user;
    }
}

// Estrategia C: GitHub (OAuth)
class GithubOAuthStrategy implements AuthStrategy {
    public function authenticate(array $credentials): ?User {
        $code = $credentials['code'];
        // ... Lógica similar a la de Google, pero con la API de GitHub ...
        $userData = $this->githubApiService->getUserProfile($code);
        $user = User::findOrCreateFromGithub($userData);
        return $user;
    }
}

```

3. El Contexto

El `LoginController` decide qué estrategia usar basándose en la ruta o un parámetro.

PHP

// Context

```

class LoginController {

    public function handleLogin() {
        $strategy = $this->getStrategyFromRequest(); // Decide la estrategia

        $user = $strategy->authenticate($_REQUEST);

        if ($user) {
            // ... Iniciar sesión ...
            // ¡Aquí podríamos iniciar el flujo 2FA que vimos antes!
            echo "Login primario exitoso. Ahora ingrese su 2FA.";
        }
    }
}

```

```

    } else {
        echo "Falló la autenticación.";
    }
}

private function getStrategyFromRequest(): AuthStrategy {
    // El usuario hizo clic en "Login con Google"
    if (isset($_GET['code']) && $_GET['state'] === 'google') {
        return new GoogleOAuthStrategy(/* ... */);
    }

    // El usuario hizo clic en "Login con GitHub"
    if (isset($_GET['code']) && $_GET['state'] === 'github') {
        return new GithubOAuthStrategy(/* ... */);
    }

    // El usuario envió el formulario de email/contraseña
    if (isset($_POST['email'])) {
        return new PasswordStrategy();
    }

    throw new Exception('Método de login no reconocido');
}
}

```

Conclusión

El patrón **Strategy** es fundamental para la autenticación porque te permite seguir el **Principio Abierto/Cerrado**: tu **LoginManager** (Contexto) está *cerrado a modificaciones*, pero está *abierto a extensiones* (puedes agregar nuevos métodos 2FA o de Login simplemente creando una nueva clase de estrategia, sin tocar el código del Contexto).